

List

24 July 2026 11:37

The data type list is an ordered sequence which is mutable and made up of one or more elements. Unlike a string, which consists of only characters, a list can have elements of different data types such as integer, float, string, tuple, or even another list. A list is a very useful to group together elements of mixed data types. Elements of a list are enclosed in brackets. And are separated by comma(,). Like string indices, list indices also starts from zero. Example:

```
#list1 is the list of 10 even numbers
list1 = [2,4,6,8,10,12,14,16,18,20]
print(list1)
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

# list2 is List of vowels.
list2 = ['a','e','i','o','A','E', 'I','O','U']
print(list2)
['a', 'e', 'i', 'o', 'A', 'E', 'I', 'O', 'U']

#list3 is list of mixed values
list3 = [100.23, 500, 'Hello',[1,2,3], 5+2j]
print(list3)
[100.23, 500, 'Hello', [1, 2, 3], (5+2j)]

#list4 is list of lists.
list4 = [['Physic', 101], ['Chemistry', 202], ['Maths', 303]]
print(list4)
[['Physic', 101], ['Chemistry', 202], ['Maths', 303]]
```

Accessing Elements in a List

The elements of the list are exist in the same way as characters are accessed in the string.

```
print(list4[0]) # Returns first element of list4.
['Physic', 101]
print(list4[-1]) # Returns last element of list 4.
['Maths', 303]

print(list3[0])
100.23
print(list3[-1])
(5+2j)

list1
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
list1[4]
10
```

List are mutable

In Python, lists are mutable. It means that the contents of the list can be changed after it has been created.

```
list1
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
list1[4]
10
list1[4] = list1[4]*2
list1[4]
20
list1
[2, 4, 6, 8, 20, 12, 14, 16, 18, 20]
```

List Operations

The data type list allows manipulation of its contents through various operations as shown below:

1. **Concatenation:** Python allows us to join two or more list using the concatenation operator depicted by the symbol plus(+). For example:

```
list1
[2, 4, 6, 8, 20, 12, 14, 16, 18, 20]
list5 = [13, 15, 17, 19]
list6 = list1 + list5
list6
[2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19]
```

2. **Repetition:** Python allows us to replicate a list using repetition operator depicted by symbol *. For example:

```
list5 = [13, 15, 17, 19]
list7 = list5*3
list7
[13, 15, 17, 19, 13, 15, 17, 19, 13, 15, 17, 19]
```

3. **Membership:** Like strings, the membership operators 'in' and checks if the element is present in the list and returns True, else returns False.

For example:

```
list6
[2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19]
11 in list6
False
12 in list6
True
```

The 'not in' operator returns True if the element is not present in the list, else it returns False.

For example:

```
list6
[2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19]
11 not in list6
True
12 not in list6
False
```

Slicing: Like strings, the slicing operation can also be applied to list.

For example:

```
list6
[2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19]
```

```

list6[:5]
[2, 4, 6, 8, 20]
list6[3:5]
[8, 20]
list6[-6:-2]
[18, 20, 13, 15]
list6[-6:-2:-2]
[]
list6[-6:-2:2]
[18, 13]
list6[-2:-6:-2]
[17, 13]
#Reversing the list
list6[::-1]
[19, 17, 15, 13, 20, 18, 16, 14, 12, 20, 8, 6, 4, 2]
#Alternate values
list6[::2]
[2, 6, 20, 14, 18, 13, 17]

```

Traversing a list

We can access each element of the list or traverse or list using a for loop or a while loop.

A. List traversal using for loop

```

count = 0
list6 = [2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19]
for e in list6:
    if e % 3 == 0 or e % 5 == 0:
        print(e, end = ' ')
        count += 1
print(f'\nnumber of values divisible by 3 or 5 is {count}')

```

Output:

```

6 20 12 18 20 15
number of values divisible by 3 or 5 is 6

```

B. List traversal using while loop

```

count = 0
list6 = [2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19]
index = 0
while index < len(list6):
    if list6[index] % 3 == 0 or list6[index] % 5 == 0:
        print(list6[index], end = ' ')
        count += 1
    index += 1
print(f'\nnumber of valueindex += 1s divisible by 3 or 5 is {count}')

```

Output:

```

6 20 12 18 20 15
number of values divisible by 3 or 5 is 6

```

List methods and built in Functions

The data type List has several built in methods that are useful in programming.

Built-in function for list manipulation

Method	Description	Example
len()	Returns the length of the list passed as the argument.	<pre>list6 = [2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19] len(list6) 14</pre>
list()	Create an empty list if no argument is passed. Creates a list if a sequence is passed as an argument.	<pre>li1 = list() li1 [] len(li1) 0 li1 = list('aeiou') li1 ['a', 'e', 'i', 'o', 'u'] len(li1) 5</pre>
append()	Appends a single element passed as an argument at the end of the list. The single element can also be a list.	<pre>>>> list1 = [10,20,30,40] >>> list1.append(50) >>> list1 [10, 20, 30, 40, 50] >>> list2 = [70, 90] >>> list1.append(list2) >>> list1 [10, 20, 30, 40, 50, [70, 90]] >>> for e in list1: ... print(e) ... 10 20 30 40 50 [70, 90] >>> list1[5] [70, 90] >>> list1[5][0] 70 >>> list1[5][1] 90 >>></pre>

extends()	Appends each element of the list passed as argument to the end of the given list.	<pre>>>> list3 [10, 20, 30, 40, 50] >>> list2 [70, 90] >>> list3.extend(list2) >>> list3 [10, 20, 30, 40, 50, 70, 90]</pre>
insert()	Inserts an element at a particular index in the list.	<pre>>>> list3 [10, 20, 30, 40, 50, 70, 90] >>> list3.insert(5, 60) >>> list3 [10, 20, 30, 40, 50, 60, 70, 90] >>></pre>
count()	Returns the number of times a given element appear in the list.	<pre>>>> list4 = [10, 20, 30, 10, 10, 20, 50] >>> list4.count(10) 3 >>> list4.count(100) 0 >>> list4.count(20) 2</pre>
index()	Returns index of the first occurrence of the element in the list. If the element is not present, ValueError is generated.	<pre>>>> list4 [10, 20, 30, 10, 10, 20, 50] >>> list4.index(30) 2 >>> list4.index(40) Traceback (most recent call last): File "<stdin>", line 1, in <module> list4.index(40) ~~~~~^ ValueError: list.index(x): x not in list >>></pre>
remove()	Remove the given element from the list. If the element is present multiple times, only the first occurrence is removed. If the element is not present, then ValueError is generated	<pre>>>> list4 [10, 20, 30, 10, 10, 20, 50] >>> list4.remove(10) >>> list4 [20, 30, 10, 10, 20, 50] >>> list4.remove(10) >>> list4 [20, 30, 10, 20, 50] >>> list4.remove(100) Traceback (most recent call last): File "<stdin>", line 1, in <module> list4.remove(100) ~~~~~^ ValueError: list.remove(x): x not in list >>></pre>
pop()	Returns the element whose index is passed as parameter to this function and also	<pre>>>> list4 [20, 30, 10, 20, 50, 70, 90, 10, 20, 30, 40, 50, [70, 90]]</pre>

	removes it from the list. If no parameter is given, then it returns and removes the last element of the list.	<pre>>>> list4.pop() [70, 90] >>> list4 [20, 30, 10, 20, 50, 70, 90, 10, 20, 30, 40, 50] >>> list4.pop(2) 10 >>> list4 [20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50] >>> list4.pop(20) Traceback (most recent call last): File "<stdin>", line 1, in <module> list4.pop(20) ~~~~~^^^^ IndexError: pop index out of range</pre>
reverse()	Reverses the order of elements in the given list.	<pre>>>> list4 [20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50] >>> list4.reverse() >>> list4 [50, 40, 30, 20, 10, 90, 70, 50, 20, 30, 20] >>></pre>
sort()	Sorts the element of the given list in place.	<pre>>>> list4 [50, 40, 30, 20, 10, 90, 70, 50, 20, 30, 20] >>> list4.sort() >>> list4 [10, 20, 20, 20, 30, 30, 40, 50, 50, 70, 90] >>> >>> animalList = ['Tiger', 'Zebra', 'Lion', 'Giraffe', 'Cat', 'Leapord'] >>> animalList.sort() >>> animalList ['Cat', 'Giraffe', 'Leapord', 'Lion', 'Tiger', 'Zebra']</pre>
sorted()	It takes a list as a parameter and creates a new list consisting of the same elements arranged in sorted order.	<pre>>>> list5 = [20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50] >>> list5 [20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50] >>> list6so = sorted(list5) >>> list6so [10, 20, 20, 20, 30, 30, 40, 50, 50, 70, 90] >>> list5 [20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50] >>></pre>

min()	Returns minimum or smallest element of the list.	>>> list5 [20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50]
max()	Returns maximum or largest element of the list.	>>> min(list5) 10 >>> max(list5) 90
sum()	Returns sum of the elements of the list.	>>> sum(list5) 430

Nested Lists: When a list appears as an element of another list, it is called a nested list. For example:

```
>>> list1
[10, 20, 30, 40, 50, [70, 90]]
```

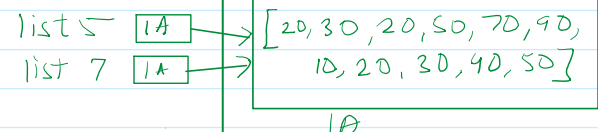
To access the element of the nested list of list one we have to specify two indices list1 [j][j]. The first index 'i' will take us to the desired nested list and second index 'j' will take us to desired element in that nested list.

Copying List

Given a list, the simplest way to make the copy of the list is to assign it to another list.

```
>>> list7 = list5
>>> list7
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50]
>>> list5
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50]
```

← meaning list 7 & list 5 points to same memory location



```
>>> list7.append(100)
>>> list7
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100]
>>> list5
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100]
```

The statement list7 = list5.

Does not create a new list. Rather it just make list5 and List7 refer to the same list object. Here list7 actually becomes an alias of list5. Therefore, any changes made to either of them will be reflected in the other list.

We can also create a copy or clone of the list as a distinct object by three methods. The first method uses slicing, the second method uses built in function list () And the third method uses copy () function of Python Library.

Method-1:

```
>>> list8 = list5[:]
>>> list8
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100]
>>> list5
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100]
```

```
>>> list8.append(200)
>>> list8
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100, 200]
>>> list5
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100]
```

Method-2:

```
>>> list9 = list(list5)
>>> list5
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100]
>>> list9
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100]
>>> list9.extend([1,2,5])
>>> list9
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100, 1, 2, 5]
>>> list5
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100]
```

Method-3:

```
>>> import copy
>>> list10 = copy.copy(list5)
>>> list10
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100]
>>> list5
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100]
>>> list10.append(230)
>>> list10
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100, 230]
>>> list5
[20, 30, 20, 50, 70, 90, 10, 20, 30, 40, 50, 100]
>>>
```

List as Argument to a Function

Whenever a list is passed as an argument to a function, we have to consider two scenarios:

- a. Elements of the original list may be changed, that is, changes made to the list in the function are reflected back in the calling function. For example, in the following program list, list one of numbers is passed as an argument to the function. Increment () This function increases every element of the list by 5.

```
# Program to increment the elements of a list.
# The list is passed as an argument to the function and increment value
also.
def increment(li1, incr):
    for i in range(len(li1)):
        li1[i] += incr
#main code
list1 = [10, 20, 30, 40, 50]
print(f'list before method call: {list1}')
i = int(input('Enter the increment value: '))
increment(list1, i)
```



```
print(f'list after method call: {list1}')
```

Output:

```
list before method call: [10, 20, 30, 40, 50]
Enter the increment value: 5
list after method call: [15, 25, 35, 45, 55]
```

- b. If the list is assigned a new value inside the function, then the new list object is created and it becomes the local copy of the function. Any changes made inside the local copy of the functions are not reflected back to the calling function. For example:

```
# Program to increment the elements of a list.
# The list is passed as an argument to the function and increment value
also.
def increment(li1, incr):
    li2 = list(li1) #local copy created
    for i in range(len(li2)):
        li2[i] += incr
    print(f'list inside method call: {li2}')
```

```
#main code
list1 = [10, 20, 30, 40, 50]
print(f'list before method call: {list1}')
```

```
i = int(input('Enter the increment value: '))
increment(list1, i)
print(f'list after method call: {list1}')
```

Output:

```
list before method call: [10, 20, 30, 40, 50]
Enter the increment value: 5
list inside method call: [15, 25, 35, 45, 55]
list after method call: [10, 20, 30, 40, 50]
```

Homework:

List Manipulation: Write a menu driven program to perform various list operations such as:

- ☐ Append an element.
- ☐ Insert an element.
- ☐ Append a list to a given list.
- ☐ Modify an existing element.
- ☐ Delete an existing element from its position.
- ☐ Delete an existing element with a given value.
- ☐ Sort the list in ascending order.
- ☐ Sort the list in descending order.
- ☐ Display the list.

